

Prowadzący: Dr. **Maria Kamińska-Zabierowska**

Symulacja komputerowa

Sprawozdanie

SYMULACJI I PORÓWNANIA SYSTEMÓW PRZYCHODNI W
DWÓCH WARIANTACH tj. WALK-IN I Z UMÓWIENIAMI

Miłosz Jasek, Łukasz Nowak

Nr albumu: **279952, 284042**

Kierunek: **Inżynieria systemów**

1. Wstęp

a. Opis słowny systemu

Przychodnia z jedną rejestracją (wliczoną w czas obsługi) i c ilością gabinetów. Rozważamy dwa tryby organizacji:

- A. **Umówienia** – pacjenci mają przydzielone rezerwacje co s minut, występuje szansa, że klient się nie zjawi (**No-show**). Czekaający klient wchodzi tylko do ustalonego gabinetu.
- B. **Walk-in** – pacjenci przychodzą w odstępach czasu losowanych z rozkładu wykładniczego o intensywności λ . Czekaający klient wchodzi dowolnego wolnego gabinetu; Jedna kolejka (**FIFO**)

b. Parametry systemu

W obu systemach

Opis parametru	Charakter (D/L)	Zakres wartości	Rozkład p-stwa
Oczekiwany czas obsługi	Deterministyczny	Liczba naturalna	-
Faktyczny czas obsługi	Losowy	Liczba rzeczywista $\geq \text{odcięcie}$	$\mathcal{N}(\mu, \sigma^2)$ odcięty z dołu
Ilość gabinetów	Deterministyczny	Liczba naturalna	-
Godziny pracy (Długość symulacji)	Deterministyczny	Liczba naturalna	-

Rys 1. Parametry występujące w obu systemach.

System z umówieniami

Opis parametru	Charakter (D/L)	Zakres wartości	Rozkład p-stwa
Współczynnik no-show	Losowy	$\{0,1\}$	$\{0.1, 0.2, 0.3\}$

Rys 2. Parametry występujące w systemie z umówieniami.

System „walk-in”

Opis parametru	Charakter (D/L)	Zakres wartości	Rozkład p-stwa
intensywność przyjść (pacjenci/h)	Losowy	{8,10,12}/h	$Exp\left(x; \frac{1}{\lambda}\right)$

Rys 3. Parametry występujące w systemie „walk-in”.

c. Cel prowadzenia symulacji

Określenie, który wariant systemu i parametry, minimalizują czas oczekiwania i maksymalizują przepustowość.

d. Wskaźniki oceny działania systemu

- Czas oczekiwania na wizytę
- Ilość obsłużonych pacjentów
- Długość kolejki
- Wykorzystanie godzin pracy lekarzy
- Ilość utraconych klientów (no-show, nie obsłużeni na końcu dnia)

e. Hipotezy badawcze

H0 – Przychodnie mają ten sam czas oczekiwania

H1 – Przychodnia w wariacie z umawianiem ma średnio krótsze czasy oczekiwania na wizytę.

H2 – Średni czas oczekiwania jest niższy w systemie z umawianiem przy liczbie gabinetów = $\mathbf{c}$ i obciążeniu = λ

2. Model i dokumentacja kodu

```
In [ ]: import matplotlib
matplotlib.use('TkAgg') # or 'Qt5Agg'
import numpy as np
import simpy
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
from pomocnicze_funkcje import readable_time, randomized_service_time, stats, print_statistics

#Naprawa importów pomocniczych funkcji po edycji ich kodu
import importlib
import pomocnicze_funkcje
importlib.reload(pomocnicze_funkcje)
```

Każdy obiekt w naszej symulacji jest klasą, która przechowuje informacje o sobie.

Dla pacjenta są to:

- `day` : identyfikator uruchomionej symulacji
- `id` : identyfikator Pacjenta
- `room` : gabinet, który obsłużył Pacjenta
- `arrival_time` : godzina, o której przyszedł
- `service_start_time` : godzina, o której wszedł do gabinetu (rozpoczęła się obsługa)
- `service_end_time` : godzina, o której wyszedł z gabinetu (zakończenie obsługi)

Każda zmienna `time` jest przechowywana jako liczba minut od rozpoczęcia symulacji

```
In [ ]: class Patient:
    def __init__(self, id):
        self.day = 1
        self.id = id
        self.room = None
        self.arrival_time = None
        self.service_start_time = None
        self.service_end_time = None
```

Dla gabinetu są to:

- `id` : identyfikator gabinetu
- `patients_served` : liczba obsłużonych pacjentów

```
In [ ]: class Room:
    def __init__(self, id):
        self.id = id
        self.patients_served = 0
```

Parametry kliniki:

- `planned_service_time` : zaplanowany czas obsługi pacjenta [minuty]
- `lambda_per_hour` : ile pacjentów przychodzi co godzinę [pacjent/60] -> zamieniamy na minuty
- `number_of_rooms` : liczba gabinetów
- `sim_time` : długość symulacji [minuty]

Pomocnicze zmienne:

- `seed` : seed generatora liczb pseudolosowych
- `list_rooms` : lista gabinetów o długości `number_of_rooms`
- `rooms` : struktura Store z biblioteki simpy o wielkości `number_of_rooms`
- `queue_len_dict` : słownik przechowujący {godzinę_zmiany:długość_kolejki}
- `queue_array` : lista przechowująca czekających pacjentów
- `last_queue_length` : ostatnia długość kolejki - do aktualizacji słownika tylko gdy zmienia się długość kolejki
- `processed_patients` : lista obsłużonych już pacjentów

Zmienna `randomized` jest tylko na poczet przypadku deterministycznego niżej i zmienia prawdziwy czas obsługi (Zamiast rozkładu normalnego używamy `planned_service_time`)

```
In [ ]: class Clinic:
    def __init__(self, planned_service_time, lambda_per_hour=10/60, seed=None, number_of_rooms=3, sim_time=480, randomized=True):
        # Inicjalizacja zmiennych symulacji
        self.curr_patient_id = 1
        self.env = simpy.Environment()
        self.randomized = randomized

        # Inicjalizacja parametrów kliniki
        self.planned_service_time = planned_service_time # w minutach
        self.lambda_per_hour = lambda_per_hour # średnia liczba przyjęć na godzinę [pacjent/60min]
        self.list_rooms = [Room(id=i + 1) for i in range(number_of_rooms)]
        self.rooms = simpy.Store(self.env, capacity=number_of_rooms)
        self.sim_time = sim_time # w minutach

        # Seed jest tutaj ustawiany, aby każdy kolejny dzień używał kolejnego stanu generatora.
        self.seed = seed
        if self.seed:
            np.random.seed(self.seed)

        # Logika statystyk kolejki
        self.queue_len_dict = {}
        self.queue_array = []
        self.last_queue_length = 0

        # Inicjalizacja pokoi w klinice
        for room in self.list_rooms:
            self.rooms.put(room)

        self.processed_patients = []
```

Metoda `update_queue_len_dict()`

Funkcja aktualizuje słownik kolejki TYLKO jeśli długość kolejki się zmieniła

```
In [ ]: def update_queue_len_dict(self, curr_time):
        current_length = len(self.queue_array)

        # Aktualizuj tylko jeśli długość kolejki się zmieniła
        if current_length != self.last_queue_length:
            self.queue_len_dict[curr_time] = current_length
            self.last_queue_length = current_length

Clinic.update_queue_len_dict = update_queue_len_dict
```

Metoda `time_between_new_patients()`:

Ważna metoda zwracająca przerwę w minutach, do pojawienia się następnego pacjenta, podawana jest jako parametr metody `env.timeout()` po wygenerowaniu pacjenta.

Wykorzystuje rozkład wykładniczy o intensywności $\frac{1}{\lambda}$

```
In [ ]: def time_between_new_patients(self):
        return np.random.exponential(1/self.lambda_per_hour) # w minutach

Clinic.time_between_new_patients = time_between_new_patients
```

Metoda `generate_patients()`:

Metoda wygeneruje pacjenta o jakimś `id` i rozpocznie dla niego proces `serve_patient()`, aż do momentu `{service_time}` przed zamknięciem. Dzięki niej upłynie czas do przyścia następnego pacjenta

```
In [ ]: def generate_patients(self, verbose=False):
        while True:
            # Pacjenci nie przychodzą {service_time} przed zamknięciem
            if self.env.now >= self.sim_time-self.planned_service_time:
                break
            patient = Patient(id=self.curr_patient_id)
            patient.arrival_time = self.env.now
            if verbose:
                print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} przybył do kliniki")
            self.env.process(self.serve_patient(patient, verbose=verbose))
            self.curr_patient_id += 1
            yield self.env.timeout(self.time_between_new_patients())

Clinic.generate_patients = generate_patients
```

Metoda `serve_patient(patient)`

Odpalana dla jakiegoś pacjenta:

- Dodaje go do kolejki
- Sprawdza czy jeszcze można obsłużyć pacjenta - jeśli mija godzina `{service_time}` od końca pracy (`sim_time`) wtedy pacjenci są usuwani z kolejki.
- Sprawdza czy jakiś pokój jest wolny i dodaje go do kolejki (Prawdziwa kolejka jest wbudowana w `SimPy` i uruchamiana jest przez `rooms.get()`)
- Obsługuje pacjentów i puszcza czas dalej.
- Zapisuje kiedy i jaki pacjent wyszedł z gabinetu
- Zwraca gabinet do puli

```
In [ ]: def serve_patient(self, patient, verbose=False):

        # Pacjent wchodzi do kolejki
        self.queue_array.append(patient.id)
        self.update_queue_len_dict(curr_time=self.env.now)

        # Funkcja w pomocnicze_funkcje.py
        if self.randomized:
            real_service_time = randomized_service_time(mean=self.planned_service_time, std=2, minimal_time=5)
        else:
            real_service_time = self.planned_service_time

        # Nie zdążyto obsłużyć pacjenta przed zamknięciem
        if self.env.now >= self.sim_time-self.planned_service_time:
            if verbose:
                print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} - nie zdążył zostać obsłużony przed zamknięciem kliniki")
            self.queue_array.remove(patient.id)
            self.update_queue_len_dict(curr_time=self.env.now)
            return

        # POBRANIE GABINETU - czytaj Pacjent wchodzi do kolejki do gabinetu
        room = yield self.rooms.get()

        # Dalsza część wykonuje się po wyjściu z kolejki, czyli po rozpoczęciu obsługi

        # Klient opuszcza kolejkę
        self.queue_array.remove(patient.id)
        self.update_queue_len_dict(curr_time=self.env.now)

        # Zaczyna się obsługa pacjenta
        patient.service_start_time = self.env.now
        if verbose:
            print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} wchodzi do gabinetu {room.id} ")

        # Mija czas obsługi
        yield self.env.timeout(real_service_time)

        # Zapisujemy kiedy wychodzi z gabinetu
        patient.service_end_time = self.env.now
        room.patients_served += 1
        if verbose:
            print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} wychodzi z gabinetu {room.id}, czekał od {readable_time(patient.arrival_time)} do {readabl

        # Przypisanie jaki gabinet obsługiwał pacjenta
        patient.room = room.id
        self.processed_patients.append(patient)

        # ZWROT GABINETU - czytaj Pacjent opuszcza gabinet
        yield self.rooms.put(room)
```

```
Clinic.serve_patient = serve_patient
```

Metoda `run()`

Uruchamia symulację, rozpoczyna proces generowania pacjentów i wyciąga dane z odpowiednich list do DataFrame-ów.

```
In [ ]: def run(self, day=1, verbose=False):
        if verbose:
            print(self.env.now)

        # Rozpoczęcie generowania pacjentów
        self.env.process(self.generate_patients(verbose=verbose))

        # UWAGA: env.run to nie to samo co ta funkcja (run)! env.run uruchamia symulację, run() to metoda klasy Clinic.
        self.env.run(until=self.sim_time)

        # Wyciągnięcie danych o obsłużonych pacjentach do DataFrame
        df = pd.DataFrame([
            {
                "day": day,
                "id": p.id,
                "room": p.room,
                "arrival_time": p.arrival_time,
                "service_start_time": p.service_start_time,
                "service_end_time": p.service_end_time,
                "service_time": p.service_end_time - p.service_start_time,
                "waiting_time": p.service_start_time - p.arrival_time
            }
            for p in self.processed_patients
        ])

        # Wyciągnięcie danych o długości kolejki do DataFrame
        df_kolejka = pd.DataFrame([
            {
                "day": day,
                "time": time,
                "queue_length": length
            }
            for time, length in self.queue_len_dict.items()
        ])

        return df.set_index(["day", "id"]), df_kolejka.set_index(["day", "time"])
Clinic.run = run
```

Metoda `reset()`

Resetuje informacje między symulacjami (dniami).

```
In [ ]: def reset(self): # Resetuje stan kliniki do początkowego między dniami symulacji
        self.env = simpy.Environment()
        self.curr_patient_id = 1
        self.list_rooms = [Room(id=i + 1) for i in range(len(self.list_rooms))]
        self.rooms = simpy.Store(self.env, capacity=len(self.list_rooms))
        for room in self.list_rooms:
            self.rooms.put(room)
        self.processed_patients = []

        # Reset statystyk kolejki: WAŻNE! (inaczej DF kolejki ma 64 MILIONY rekordów i waży 2 GB)
        self.queue_len_dict = {}
        self.queue_array = []
        self.last_queue_length = 0

Clinic.reset = reset
```

Metoda `run_multiple_times`

Tworzy nowe środowisko dla każdego uruchomienia i łączy wyniki do DataFrame'a `multiple_df`

Stworzenie nowego środowiska nie zmienia seeda ani stanu generatora liczb - każdy następny dzień używa następnego stanu generatora.

```
In [ ]: def run_multiple_times(self, n, verbose=False):
        list_dfs = []
        list_dfs_queue = []
        for i in range(1, n+1):
            if verbose:
                print(f"-----{i}-----")
            self.reset()
            single_df = self.run(i, verbose=verbose)
            list_dfs.append(single_df[0])
            list_dfs_queue.append(single_df[1])

        # Szybsze scalać wyniki (Lepiej zmienić listę na DF niż ciągle doklejać)
        multiple_df = pd.concat(list_dfs)
        multiple_df_queue = pd.concat(list_dfs_queue)
        return multiple_df, multiple_df_queue

Clinic.run_multiple_times = run_multiple_times

In [ ]: clinic = Clinic(number_of_rooms=3, planned_service_time=15, sim_time=480, lambda_per_hour=10/60, seed=42, randomized=True)

In [ ]: df, df_kolejka = clinic.run_multiple_times(n=1000, verbose=False)

In [ ]: df.to_csv("../wyniki/wyniki_bez_um.csv")
        df_kolejka.to_csv("../wyniki/wyniki_kolejka_bez_um.csv")

In [ ]: stats(df, df_kolejka, service_minutes=15, total_time_hours=8)

In [ ]: # histogram obsłużonych pacjentów (kubетки ile pacjentów na dzień)
        num_patients_per_day = df.groupby('day').size()

In [ ]: plt.figure(figsize=(10, 6))
        sns.histplot(num_patients_per_day, bins=15, kde=True)
        plt.title('Histogram liczby obsłużonych pacjentów na dzień')
```

```
plt.xlabel('Liczba pacjentów na dzień')
plt.ylabel('Liczba dni')
plt.show()

print_statistics(num_patients_per_day, "liczba obsłużonych pacjentów na dzień")
```

```
In [ ]: #histogram straconych pacjentów na zamknięcie (ostatni moment z dnia przed zamknięciem)
patients_lost_per_day = df_kolejka.groupby('day').tail(1)['queue_length']

plt.figure(figsize=(10, 6))
sns.histplot(patients_lost_per_day, bins=15, kde=True, color='coral')
plt.title('Histogram liczby straconych pacjentów na zamknięcie')
plt.xlabel('Liczba pacjentów w kolejce na zamknięcie')
plt.ylabel('Liczba dni')
plt.show()

# Statystyki
print_statistics(patients_lost_per_day, "liczba straconych pacjentów na zamknięcie")
```

Prosty przykład (deterministyczny)

Opis zasad:

- Dla ułatwienia obliczeń pacjenci przychodzą w deterministycznych, równych odstępach czasu
- Przychodzi 6 pacjentów na godzinę (czyli przychodzą w odstępach 10 minut)
- czas trwania symulacji = 2 godziny
- czas jednej wizyty = 15 minut
- są dwa gabinety

Czas	Zdarzenia	start	koniec
8:00	Pacjent 1 wchodzi do Gabinetu 1	8:00	8:15
8:10	Pacjent 2 wchodzi do Gabinetu 2	8:10	8:25
8:15	Pacjent 1 zwalnia Gabinet 1	-	-
8:20	Pacjent 3 wchodzi do Gabinetu 1	8:20	8:35
8:25	Pacjent 2 zwalnia Gabinet 2	-	-
8:30	Pacjent 4 wchodzi do Gabinetu 2	8:30	8:45
8:35	Pacjent 3 zwalnia Gabinet 1	-	-
8:40	Pacjent 5 wchodzi do Gabinetu 1	8:40	8:55
8:45	Pacjent 4 zwalnia Gabinet 2	-	-
8:50	Pacjent 6 wchodzi do Gabinetu 2	8:50	9:05
8:55	Pacjent 5 zwalnia Gabinet 1	-	-
9:00	Pacjent 7 wchodzi do Gabinetu 1	9:00	9:15
9:05	Pacjent 6 zwalnia Gabinet 2	-	-
9:10	Pacjent 8 wchodzi do Gabinetu 2	9:10	9:25
9:15	Pacjent 7 zwalnia Gabinet 1	-	-
9:20	Pacjent 9 wchodzi do Gabinetu 1	9:20	9:35
9:25	Pacjent 8 zwalnia Gabinet 2	-	-
9:30	Pacjent 10 wchodzi do Gabinetu 2	9:30	9:45
9:35	Pacjent 9 zwalnia Gabinet 1	-	-
9:40	Pacjent 11 wchodzi do Gabinetu 1	9:40	9:55
9:45	Pacjent 10 zwalnia Gabinet 2	-	-
9:45	Koniec przyjmowania pacjentów.	-	-
9:55	Pacjent 11 zwalnia Gabinet 1	-	-
10:00	Koniec symulacji	-	-

Podczas tej symulacji

Przyjmujemy deterministyczny przypadek przez ustawienie czasu między pacjentami jako 10 minut. Jako, że nie ma tutaj losowości wystarczy pojedynczy przypadek.

```
In [ ]: def time_between_new_patients(self):
        return 10
Clinic.time_between_new_patients = time_between_new_patients

In [ ]: clinic_deterministic = Clinic(number_of_rooms=2, planned_service_time=15, sim_time=120, lambda_per_hour=6/60, randomized=False)

In [ ]: # single_df = clinic_deterministic.run(day=1)
df = clinic_deterministic.run(day=1, verbose=True)
```

0
Czas 8:00: Pacjent 1 przybył do kliniki
Czas 8:00: Pacjent 1 wchodzi do gabinetu 1
Czas 8:10: Pacjent 2 przybył do kliniki
Czas 8:10: Pacjent 2 wchodzi do gabinetu 2
Czas 8:15: Pacjent 1 wychodzi z gabinetu 1, czekał od 8:00 do 8:00
Czas 8:20: Pacjent 3 przybył do kliniki
Czas 8:20: Pacjent 3 wchodzi do gabinetu 1
Czas 8:25: Pacjent 2 wychodzi z gabinetu 2, czekał od 8:10 do 8:10
Czas 8:30: Pacjent 4 przybył do kliniki
Czas 8:30: Pacjent 4 wchodzi do gabinetu 2
Czas 8:35: Pacjent 3 wychodzi z gabinetu 1, czekał od 8:20 do 8:20
Czas 8:40: Pacjent 5 przybył do kliniki
Czas 8:40: Pacjent 5 wchodzi do gabinetu 1
Czas 8:45: Pacjent 4 wychodzi z gabinetu 2, czekał od 8:30 do 8:30
Czas 8:50: Pacjent 6 przybył do kliniki
Czas 8:50: Pacjent 6 wchodzi do gabinetu 2
Czas 8:55: Pacjent 5 wychodzi z gabinetu 1, czekał od 8:40 do 8:40
Czas 9:00: Pacjent 7 przybył do kliniki
Czas 9:00: Pacjent 7 wchodzi do gabinetu 1
Czas 9:05: Pacjent 6 wychodzi z gabinetu 2, czekał od 8:50 do 8:50
Czas 9:10: Pacjent 8 przybył do kliniki
Czas 9:10: Pacjent 8 wchodzi do gabinetu 2
Czas 9:15: Pacjent 7 wychodzi z gabinetu 1, czekał od 9:00 do 9:00
Czas 9:20: Pacjent 9 przybył do kliniki
Czas 9:20: Pacjent 9 wchodzi do gabinetu 1
Czas 9:25: Pacjent 8 wychodzi z gabinetu 2, czekał od 9:10 do 9:10
Czas 9:30: Pacjent 10 przybył do kliniki
Czas 9:30: Pacjent 10 wchodzi do gabinetu 2
Czas 9:35: Pacjent 9 wychodzi z gabinetu 1, czekał od 9:20 do 9:20
Czas 9:40: Pacjent 11 przybył do kliniki
Czas 9:40: Pacjent 11 wchodzi do gabinetu 1
Czas 9:45: Pacjent 10 wychodzi z gabinetu 2, czekał od 9:30 do 9:30
Czas 9:55: Pacjent 11 wychodzi z gabinetu 1, czekał od 9:40 do 9:40

Teoria (deterministyczna) zgadza się z symulacją

Sanity check przychodnia - wariant z umówieniami

Dokumentacja kodu

```
In [ ]: import matplotlib
matplotlib.use('TkAgg') # or 'Qt5Agg'
import numpy as np
import simpy
import matplotlib.pyplot as plt
import random
import pandas as pd
from pomocnicze_funkcje import readable_time, randomized_service_time, stats, print_statistics
import seaborn as sns

#Naprawa importów pomocniczych funkcji po edycji ich kodu
import importlib
import pomocnicze_funkcje
importlib.reload(pomocnicze_funkcje)
```

Parametry klasy Patient:

- **id**: identyfikator pacjenta
- **arrival_time**: czas przybycia pacjenta do kliniki
- **service_start_time**: czas rozpoczęcia wizyty
- **service_end_time**: czas zakończenia wizyty
- **room**: identyfikator gabinetu przypisanego pacjentowi
- **real_service_time**: przechowuje liczbę pseudolosową wygenerowaną z rozkładu normalnego odpowiadającą długości wizyty

```
In [ ]: class Patient:
    def __init__(self, id):
        self.id = id
        self.arrival_time = None
        self.service_start_time = None
        self.service_end_time = None
        self.room = None
        self.real_service_time = None
```

Parametry klasy Room:

- **id**: identyfikator gabinetu
- **env**: środowisko symulacji
- **patients_served**: ilość obsłużonych pacjentów
- **queue**: kolejka do gabinetu
- **queue_len_dict**: pozwala zapisywać długość kolejki w różnych momentach symulacji
- **resource**: wskazuje na obiekt z biblioteki simpy który odpowiada za faktyczne obsługiwanie symulacji

Oraz funkcja **update_queue_len_dict** która odpowiada za zapisywanie długości kolejki w danym momencie czasu do słownika **queue_len_dict**

```
In [ ]: class Room:
    def __init__(self, id, env):
        self.id = id
        self.patients_served = 0
        self.no_show = 0
        self.queue = []
        self.queue_len_dict = dict()
        self.resource = simpy.Resource(env, capacity=1)
    def update_queue_len_dict(self, curr_time):
        self.queue_len_dict[curr_time] = len(self.queue)
```

Parametry klasy Clinic:

- **curr_patient_id**: aktualny identyfikator pacjenta używany przy tworzeniu pacjentów
- **env**: środowisko symulacji
- **service_time**: czas obsługi jednego pacjenta
- **no_show**: prawdopodobieństwo że pacjent nie zjawi na wizytę
- **seed**: seed generatora liczby pseudolosowych
- **list_rooms**: lista gabinetów
- **processed_patients**: lista, która przechowuje informacje o pacjentach którzy już zostali obsłużeni
- **sim_time**: całkowity czas symulacji

```
In [ ]: class Clinic:
    def __init__(self, number_of_rooms, planned_service_time, no_show_proportion=0.2, seed=None, sim_time=480):
        # zapisanie parametrów
        self.env = simpy.Environment()
        self.curr_patient_id = 1
        self.planned_service_time = planned_service_time
        self.no_show_proportion = no_show_proportion
        self.sim_time = sim_time
        # utworzenie pokoi
        self.list_rooms = [Room(id=i + 1, env=self.env) for i in range(number_of_rooms)]
        self.processed_patients = []

        self.seed = seed
        if self.seed:
            random.seed(self.seed)
            np.random.seed(self.seed)
```

Metoda **generate_patients** jako wejście przyjmuje instancje klasy **room** i służy do generowania pacjentów i nadawania im atrybutów takich jak czas obsługi (**real_service_time**) generowany z rozkładu normalnego oraz gabinet z którego będzie korzystał. Pacjenci są generowani co stały czas **planned_service_time** i losuje czy osoba się pojawi czy wystąpi no show. Następnie przekazuje pacjenta do metody **serve_patient**.

Jeżeli czas symulacji będzie wyższy niż aktualnego czasu + planowanego czasu obsługi to generowanie pacjentów zostaje przerwane aby nie doszło do sytuacji w której pacjenci są obsługiwani po zakończeniu pracy przychodni

```
In [ ]: def generate_patients(self, room, verbose=False):

    def time_between_new_patients(): # funkcja podająca czas między wizytami
        return self.planned_service_time

    while True:
        if self.env.now > self.sim_time - self.planned_service_time: # Pacjenci nie przychodzą przed {planned_service_time} zamknijęciem
            break

        # tworzymy pacjenta i nadajemy mu atrybuty
        patient = Patient(id=f"{room.id}.{self.curr_patient_id}")
        patient.room = room.id
        patient.real_service_time = randomized_service_time()

        # sprawdzamy no_show
        if random.random() < self.no_show_proportion: #pacjent nie przyszedł
            if verbose:
                print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} nie pojawił w gabinecie {room.id} w ciągu {self.planned_service_time} minut")
                room.no_show += 1

            else: #pacjent przyszedł
                patient.arrival_time = self.env.now
                if verbose:
                    print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} przybył do kliniki")
                self.env.process(self.serve_patient(patient, room, verbose))
    Clinic.generate_patients = generate_patients
```

Metoda **serve_patient** służy do operowania kolejką do gabinetu samą obsługą pacjenta oraz jego wyjściem z gabientu. Zapisuje również stany kolejki.

```
In [ ]: def serve_patient(self, patient, room, verbose=False):
    # pacjent wchodzi do kolejki
    room.queue.append(patient.id)
    room.update_queue_len_dict(curr_time=self.env.now)

    with room.resource.request() as request:
        yield request
    # pacjent wychodzi z kolejki i wchodzi do gabinetu
    patient.service_start_time = self.env.now
    room.queue.remove(patient.id)
    room.update_queue_len_dict(curr_time=self.env.now)
    if verbose:
        print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} wchodzi do gabinetu {room.id} ")
    yield self.env.timeout(patient.real_service_time)
    # pacjent wychodzi z gabinetu
    patient.service_end_time = self.env.now
    room.patients_served += 1
    if verbose:
        print(f"Czas {readable_time(self.env.now)}: Pacjent {patient.id} wychodzi z gabinetu {room.id}")
    self.processed_patients.append(patient)
Clinic.serve_patient = serve_patient
```

Metoda **reset** służy do resetowania kliniki między kolejnymi wywołaniami polecenia **run** w metodzie **run_multiple_times**

```
In [ ]: def reset(self):
    self.env = simpy.Environment()
    self.curr_patient_id = 1
    self.list_rooms = [Room(id=i + 1, env=self.env) for i in range(len(self.list_rooms))]
    self.processed_patients = []
Clinic.reset = reset
```

Metoda **run** odpowiada za uruchomienie symulacji dla jednego dnia po wywołaniu dla każdego gabinetu rozpoczyna osobny generator pacjentów.

Czas trwania jest wydłużony o 0.01 aby komunikaty które pojawiały się w ostatniej "minucie" symulacji mogły się wyświetlić.

Zawiera również funkcję **sum_queue_len_data** która sumuje stany kolejki ze wszystkich gabinetów.

I ostatecznie po zakończeniu symulacji zapisuje zarówno dane o długości kolejki oraz dane o obsłużonych pacjentach do DataFrame.

```
In [ ]: def run(self, day=1, verbose=False):
    # funkcja do łączenia słowników z kolejkami w jeden DataFrame
    def sum_queue_len_data():
        all_times = set()
        for room in self.list_rooms:
            all_times.update(room.queue_len_dict.keys())

        all_times = sorted(all_times)
        queue_data = []
        last_known = {room.id: 0 for room in self.list_rooms}

        for time in all_times:
            for room in self.list_rooms:
                if time in room.queue_len_dict:
                    last_known[room.id] = room.queue_len_dict[time]

            total_queue = sum(last_known.values())
            queue_data.append({
                'day': day,
                'time': time,
                'total_queue_length': total_queue
            })
        queue_data = pd.DataFrame(queue_data)
        queue_data = queue_data.set_index(["day", "time"])
        return queue_data

    # Dla każdego pokoju uruchamiamy osobny proces generowania i obsługi pacjentów
    for room in self.list_rooms:
        self.env.process(self.generate_patients(room, verbose))
    # Symulacja trwa do ustalonego czasu + 0.01 minuty aby rekordy ostatnich pacjentów zdarzyły zostać zapisane
    self.env.run(until=self.sim_time + 0.01)
    # Zapisujemy do pandas DataFrame
    df = pd.DataFrame([
        {
            "day": day,
            "id": p.id,
```

```
        "room": p.room,
        "arrival_time": p.arrival_time,
        "service_start_time": p.service_start_time,
        "service_end_time": p.service_end_time,
        "service_time": p.service_end_time - p.service_start_time,
        "waiting_time": p.service_start_time - p.arrival_time
    }
    for p in self.processed_patients
])
# df = df.set_index("id")
queue_df = sum_queue_len_data()
return df.set_index(["day", "id"]), queue_df
Clinic.run = run
```

Metoda `run_multiple_times`

Przyjmuje `n` jako zmienną, która określa ilość wykonanych symulacji.

Łączy również dane z wszystkich wywołań `run` w jeden DataFrame.

Oraz dodatkowo zlicza ilość pacjentów ogółem, pacjentów obsłużonych, pacjentów no-show oraz proporcje no-show do wszystkich.

```
In [ ]: def run_multiple_times(self, n, verbose=False, verbose_proportions=False):
        no_show_per_day = []
        sum_no_show = 0
        sum_patients_served = 0

        for i in range(1, n + 1):
            if verbose:
                print(f"-----{i}-----")
            self.reset()
            single_df, single_queue_df = self.run(i, verbose)

            no_show_daily = 0

            for room in self.list_rooms:
                no_show_daily += room.no_show
                sum_patients_served += room.patients_served
            no_show_per_day.append({'day': i, 'no_show_amount': no_show_daily})

            sum_no_show += no_show_daily
            multiple_df = pd.concat([multiple_df, single_df]) if i > 1 else single_df
            multiple_queue_df = pd.concat([multiple_queue_df, single_queue_df]) if i > 1 else single_queue_df
        if verbose and verbose_proportions:
            print()
            if verbose_proportions:
                print(f"wszyscy pacjenci: {sum_patients_served + sum_no_show}")
                print(f"obsłużeni pacjenci: {sum_patients_served}")
                print(f"no-show pacjenci: {sum_no_show}")
                print(f"proporcja: {sum_no_show/(sum_patients_served + sum_no_show)}")

            no_show_df = pd.DataFrame(no_show_per_day)
            no_show_df = no_show_df.set_index(["day"])

        return multiple_df, multiple_queue_df, no_show_df
Clinic.run_multiple_times = run_multiple_times
```

Uruchomienie symulacji dla 1000 dni:

```
In [ ]: clinic1 = Clinic(number_of_rooms=3, planned_service_time=15, sim_time=8*60, seed=42)
df, queue_df, nshow = clinic1.run_multiple_times(n=1000)

df.to_csv("../wyniki/wyniki_z_um.csv")
queue_df.to_csv("../wyniki/kolejki_z_um.csv")
```

Uruchomienie symulacji i porównanie z wynikiem teoretycznym:

```
In [ ]: clinic1 = Clinic(number_of_rooms=2, planned_service_time=15, sim_time=120, no_show_proportion=0.4)
_, _ = clinic1.run()
```

```
In [ ]: sum_no_show = 0
sum_patients_served = 0
for room in clinic1.list_rooms:
    print(f"Gabinet {room.id}:")
    print(f"    Pacjentów obsłużonych: {room.patients_served}")
    print(f"    Pacjentów nieobecnych (no-show): {room.no_show}")
    print(f"    Łącznie pacjentów wygenerowanych: {room.patients_served + room.no_show}")
    sum_no_show += room.no_show
    sum_patients_served += room.patients_served
print()
print(f"Łącznie pacjentów obsłużonych: {sum_patients_served}")
print(f"Łącznie pacjentów nieobecnych (no-show): {sum_no_show}")
```

Symulacja "na kartce"

Opiz zasad:

- zakładamy, że czas obsługi jest deterministyczny i zawsze trwa 15 min
- szansa na no-show to 0.4
- no-show jest losowane za pomocą kostki 10d
- czas trwania symulacji to 2 h
- jest tylko 2 gabinet

Dla gabinetu 1:

Czas	Zdarzenia	Rzut kostką
8:00	Pacjent 1 nie pojawił się	2
8:15	Pacjent 2 pojawił się i wchodzi do gabinetu	10
8:30	Pacjent 2 wychodzi z gabinetu Pacjent 3 pojawił się i wchodzi do gabinetu	5

Czas	Zdarzenia	Rzut kostką
8:45	Pacjent 3 wychodzi z gabinetu Pacjent 4 pojawił się i wchodzi do gabinetu	8
9:00	Pacjent 4 wychodzi z gabinetu Pacjent 5 nie pojawił się	4
9:15	Pacjent 6 pojawił się i wchodzi do gabinetu	8
9:30	Pacjent 6 wychodzi z gabinetu Pacjent 7 pojawił się i wchodzi do gabinetu	9
9:45	Pacjent 7 wychodzi z gabinetu Pacjent 8 nie pojawił się	2
10:00	Koniec symulacji	—

Dla gabinetu 2:

Czas	Zdarzenia	Rzut kostką
8:00	Pacjent 1 pojawił się i wchodzi do gabinetu	5
8:15	Pacjent 1 wychodzi z gabinetu Pacjent 2 pojawił się i wchodzi do gabinetu	8
8:30	Pacjent 2 wychodzi z gabinetu Pacjent 3 pojawił się i wchodzi do gabinetu	9
8:45	Pacjent 3 wychodzi z gabinetu Pacjent 4 pojawił się i wchodzi do gabinetu	7
9:00	Pacjent 4 wychodzi z gabinetu Pacjent 5 nie pojawił się	1
9:15	Pacjent 6 pojawił się i wchodzi do gabinetu	6
9:30	Pacjent 6 wychodzi z gabinetu Pacjent 7 pojawił się i wchodzi do gabinetu	10
9:45	Pacjent 7 wychodzi z gabinetu Pacjent 8 pojawił się i wchodzi do gabinetu	7
10:00	Pacjent 8 wychodzi z gabinetu Koniec symulacji	—

Obsłużono 12 pacjentów, 4 pacjentów nie zjawilo się na wizycie

Model zachowuje się w sposób zbliżony do przykładu teoretycznego.

Wywołanie wielokrotne:

```
In [ ]: clinic2 = Clinic(number_of_rooms=2, planned_service_time=15)
multiple_df, queue_df = clinic2.run_multiple_times(n=100, verbose_proportions=True)
```

Proporcja no show do ilości wszystkich pacjentów jest zbliżona do 0.2 czyli wartości która oryginalnie zakładaliśmy.

```
In [ ]: multiple_df.describe()
```

3. Plan i realizacja badań

W celu porównania obu systemów organizacji stosujemy **badania monoselekcyjne** – zmienianym parametrem jest **system organizacji**. Obserwujemy jakie wartości przyjmują wskaźniki oceny działania systemu dla obu wariantów przychodni. Wartości parametrów wejściowych dobrano a priori jako reprezentatywne dla typowych warunków pracy przychodni i utrzymano stałe dla obu wariantów systemu w celu zapewnienia porównywalności wyników.

Model będzie uruchamiany 1000 razy dla parametrów:

- Intensywność przyjść klientów $\lambda = 10/h$
- Współczynnik no-show = 0.2
- Planowany czas obsługi = 15 [min]
- Faktyczny czas obsługi = $\mathcal{N}(15, 2)$ [min]
- Ilość gabinetów = 3
- Godziny pracy = 8 [h]

I dwóch różnych systemów organizacji: WALK-IN i Z UMÓWIENIAMI

Wariant z umówieniami

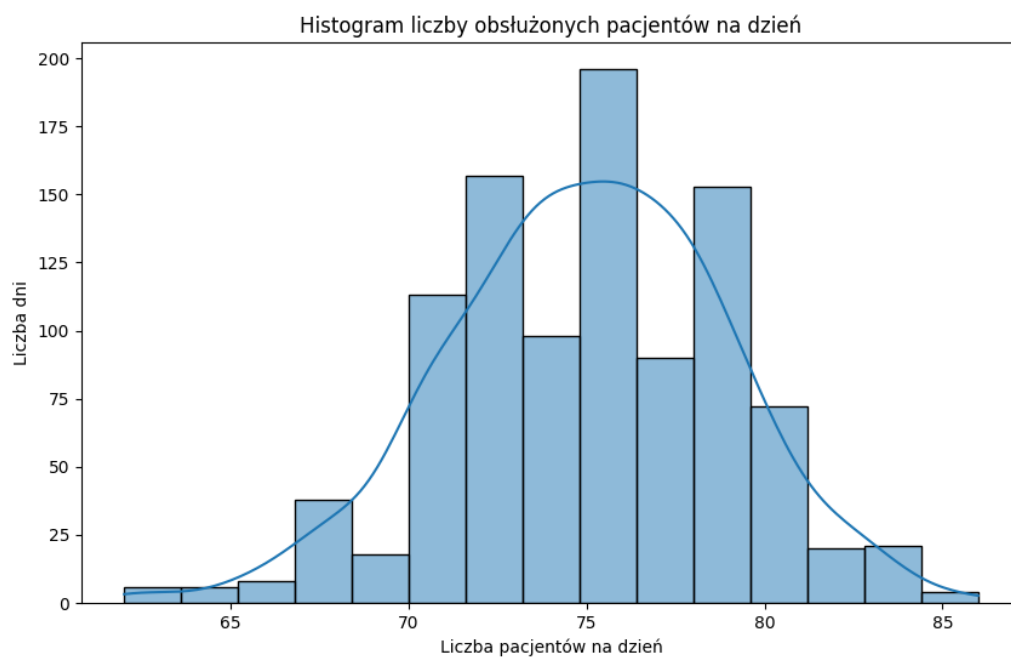
Statystyka	Czas oczekiwania na wizytę	Ilość obsłużonych pacjentów	Uśredniona dzienna długość kolejki	Wykorzystanie godzin pracy lekarzy	Ilość utraconych pacjentów
Średnia	1.63	74.97	0.27	78.01%	19.35
Odch. Standardowe	0.72	3.93	0.12	4.21%	3.99
Min	0.40	62.00	0.05	63.87%	8.00
Max	6.47	86.00	1.06	89.06%	33.00
25%	1.15	72.00	1.15	75.24%	17.00
Mediana	1.48	75.00	0.24	78.11%	19.00
95%	3.00	81.00	3.00	84.90%	26.00

Rys 4. Kluczowe wskaźniki oceny działania systemu oraz ich statystyki testowe dla systemu w wariancie z umówieniami

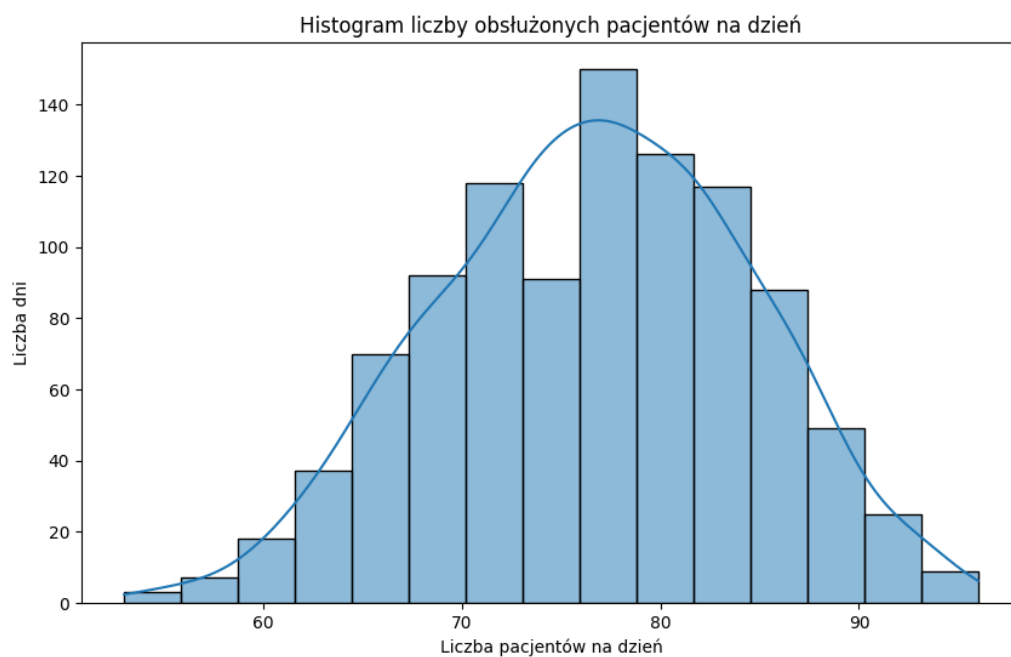
Wariant WALK-IN (bez umówień)

	Czas oczekiwania na wizytę	Ilość obsłużonych pacjentów	Uśredniona dzienna długość kolejki	Wykorzystanie godzin pracy lekarzy	Ilość utraconych pacjentów
Średnia	8.19	76.64	1.47	79.78%	0.63
Odch. Standardowe	6.58	7.86	1.35	8.20%	1.68
Min	0.59	53	0.06	56.64%	0
Max	47.49	96	10.6	98.49%	14
25%	3.99	71	0.64	74.14%	0
Mediana	6.31	77	1.05	79.91%	0
95%	22.4	89	4.25	93.10%	4

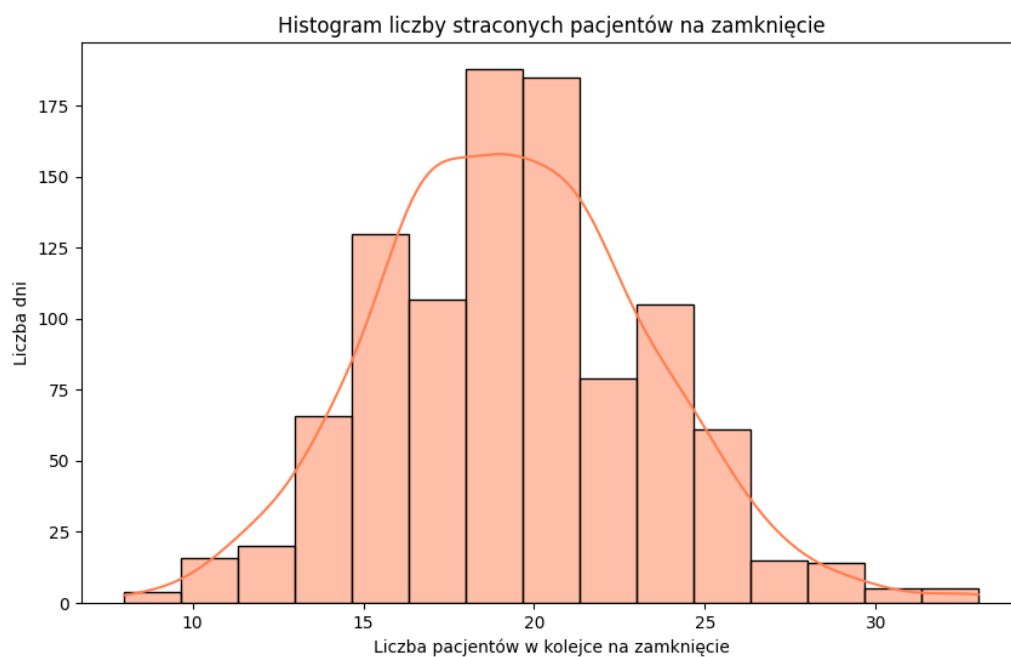
Rys 5. Kluczowe wskaźniki oceny działania systemu oraz ich statystyki testowe dla systemu w wariancie „WALK-IN” - bez umówień



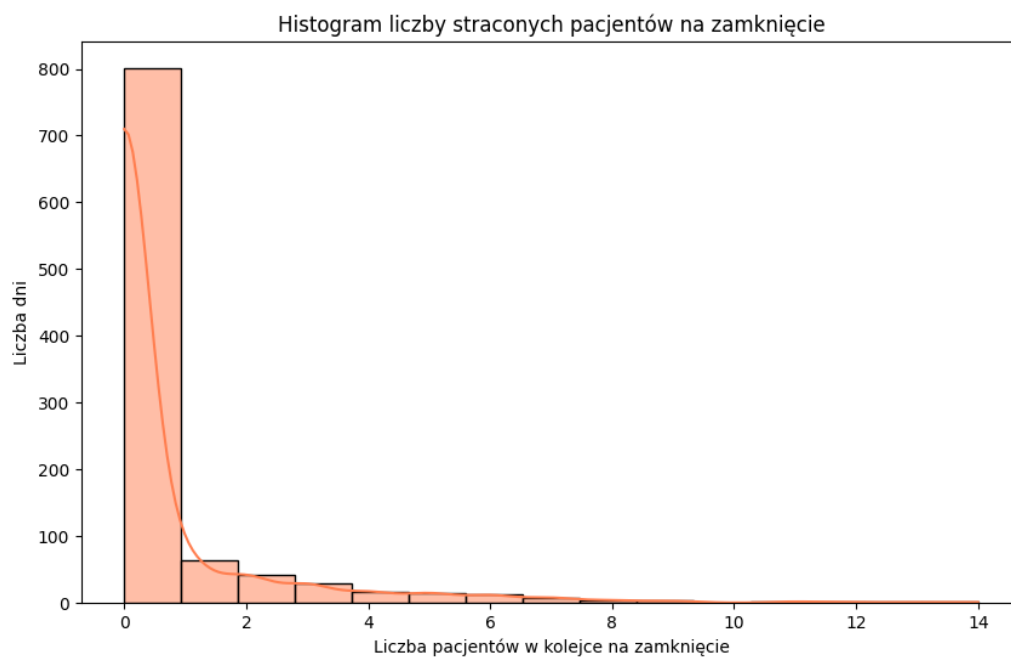
Rys 6. Histogram obsłużonych pacjentów na dzień dla systemu w wariancie **z umówieniami**



Rys 7. Histogram obsłużonych pacjentów na dzień dla systemu w wariancie „WALK-IN” - bez umówień



Rys 8. Histogram utraconych pacjentów na dzień dla systemu we wariancie **z umówieniami**



Rys 9. Histogram utraconych pacjentów na dzień dla systemu w wariancie **„WALK-IN” - bez umówieni**

4. Analiza i wnioski

a. Hipotezy statystyczne

μ – średni czas oczekiwania

$$H_0: \mu_{\text{z umówieniami}} = \mu_{\text{walk-in}}$$

$$H_1: \mu_{\text{z umówieniami}} < \mu_{\text{walk-in}}$$

Odrzucamy H_0 , gdy p nie przekroczy poziom istotności $\alpha = 0.05$:

b. Z-Test – Porównanie czasów oczekiwania

System BEZ UMAWIANIA	System Z UMÓWIENIAMI
Średnia: 8.65 [min]	Średnia: 1.64 [min]
Odchylenie std: 11.59 [min]	Odchylenie std: 2.59 [min]
Liczba obserwacji: 76641	Liczba obserwacji: 74962
Różnica średnich: $\Delta\mu = 7.01$ [min]	
Z-statystyka: 163.1680 p-value (jednostronny): 0 $\alpha = 0.05$	
ODRZUCAMY H0 ($p < 0.05$) System Z UMÓWIENIAMI ma ISTOTNIE NIŻSZE czasy oczekiwania	

Rys 10. Tabela przedstawiająca statystyki testowe i wynik **Z-testu**.

c. Spostrzeżenia i wnioski

- Przychodnia w wariancie umawianym ma zdecydowanie więcej utraconych pacjentów ze względu na nieprzychodzenie pacjentów na wizyty (no-show).
- Niski parametr $\lambda = 10 \left[\frac{\text{pacjent}}{h} \right]$ prowadził do krótkich kolejek, mimo tego miał spory czas oczekiwania przy wyższym obciążeniu spekulujemy, że system „WALK-IN” działałby z jeszcze większym czasem oczekiwania oraz dużo większymi stratami w klientach. (Jedyny sposób straty klienta w systemie „WALK-IN”
- Oba warianty przychodni są w stanie średnio obsłużyć podobną ilość pacjentów.
- Proporcja czasu, w którym lekarze przyjmują pacjentów do całego czasu ich pracy jest porównywalna w obu wariantach.
- W wariancie z umawianiami średnia dzienna długość kolejki jest zdecydowanie niższa niż w wariancie „WALK-IN”.

System Z UMÓWIENIAMI ma ISTOTNIE NIŻSZE czasy oczekiwania

Deklaracja o użyciu narzędzi sztucznej inteligencji

Przy pracy ze sprawozdaniem wykorzystane zostały modele:

- Claude Haiku 4.5 - Co-pilot - podpowiadanie syntaxu i parametrów
- ChatGPT-4.1 - Co-pilot - podpowiadanie syntaxu i optymalizacja utworzonych rozwiązań przez wychwytywanie małych bugów (błędne indexy, błędne parametry)